INF384 · TEMAS AVANZADOS EN INGENIERÍA DE SOFTWARE 1 · 2026-2

Sesión 03

Integración continua: versionamiento, librerías y build

Estrategias de ramificación, gestión de dependencias y construcción de artefactos reproducibles

Capítulo 3 (1 de 2) · 3 horas | RA2 | Prof. José Felipe del Castillo Salazar

PUCP

INF384 · 9 CAPÍTULOS · 14 SESIONES · 42 HORAS

La ruta del curso



SESIÓN 3

Objetivo de la sesión y ruta

OBJETIVO

Analizar una estrategia de versionamiento y de gestión de dependencias, y justificar su elección para un contexto organizacional dado.

NO BASTA CON ELEGIR

Hay que defender por qué esa estrategia sirve en ese contexto y no en otro. La justificación es la mitad de la respuesta.

1

Versionamiento de código

Qué es integración continua en sentido estricto y qué estrategia de ramificación la hace posible.

2

Gestión de librerías

Dependencias transitivas, builds reproducibles y el riesgo del código que no escribimos.

3

Build

Construcción de un artefacto único, versionado e inmutable, y su efecto sobre las métricas del flujo.

4

Calidad de código

Métricas de calidad y quality gate

Resultado de Aprendizaje vinculado: RA2 - diseñar e implementar pipelines de integración y despliegue continuo.

CONTINUIDAD CON LA SESIÓN 2

Dónde estamos en el pipeline



Lo que ya establecimos

- El pipeline se ordena por costo de detección: lo rápido va primero.
- La salida de la integración continua es un artefacto validado.
- Se construye una vez y se despliega muchas veces.
- Lotes pequeños reducen el riesgo, no lo aumentan.

Lo que falta responder hoy

- ¿Cómo tiene que entrar el código para que el pipeline sirva de algo?
- ¿Qué pasa con el código que no escribimos nosotros?
- ¿Qué significa exactamente construir "una sola vez"?

Un pipeline excelente sobre una estrategia de ramificación mala no produce integración continua.

A

Versionamiento de código

Qué es integración continua en sentido estricto y qué estrategia de ramificación la sostiene

CAPÍTULO 3.1

Qué es realmente integración continua

Cada integrante del equipo integra su trabajo con el de los demás al menos una vez al día, y cada integración es verificada automáticamente.

Lo que la mayoría llama CI

- Tener configurado un servidor de integración.
- Que corra algo automáticamente al abrir un pull request.
- Ver la marca verde antes de hacer merge.
- Ramas de funcionalidad que viven dos semanas y luego se integran.

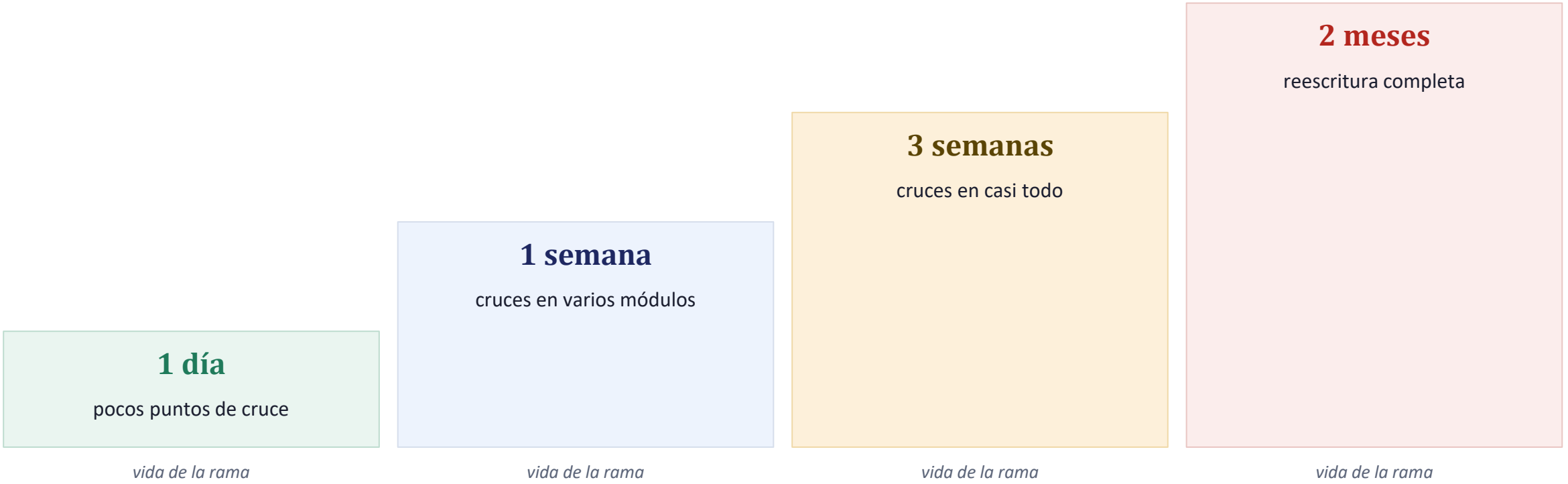
Lo que la definición exige

- Integración con el código del equipo al menos diaria, por cada persona.
- Verificación automática de cada integración, antes de unirla a la rama principal
- La rama principal siempre en estado desplegable.
- Si la verificación falla, arreglarla es la prioridad del equipo.

La distinción no es académica: determina si el pipeline detecta el conflicto cuando es de tres líneas o cuando ya es de trescientas.

POR QUÉ LA FRECUENCIA IMPORTA

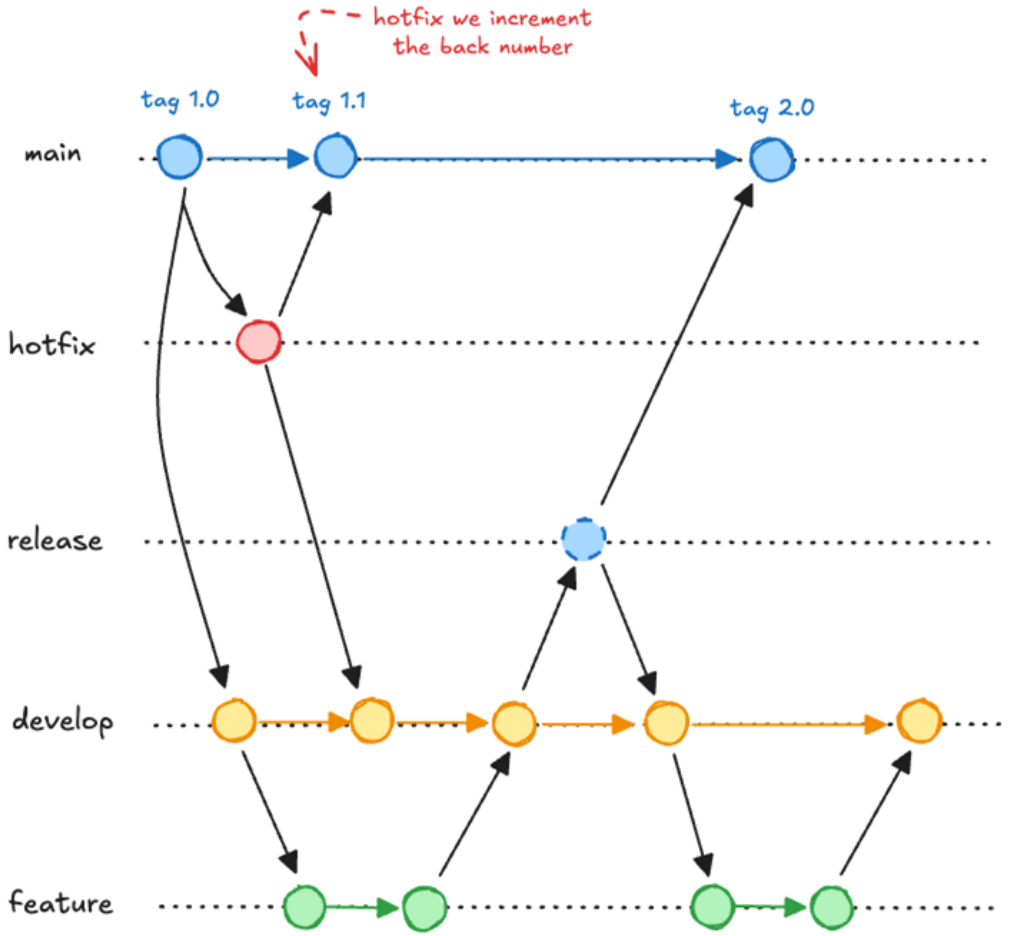
El costo de integrar tarde



El conflicto no crece con las líneas cambiadas sino con las combinaciones posibles entre cambios.
En el value stream map de la sesión 1, esto aparece como retrabajo: trabajo que nadie estimó ni planificó.

ESTRATEGIA DE RAMIFICACIÓN 1

GitFlow



ESTRATEGIA DE RAMIFICACIÓN 1

GitFlow

`main`	Solo versiones publicadas - PRODUCCIÓN
`release`	Estabilización previa a la publicación, permanente - QA/STAGING
`develop`	Integración de funcionalidades, permanente - DEVELOP
`feature/*`	Trabajo individual, vida corta - LOCAL
`hotfix/*`	Correcciones urgentes sobre lo publicado - LOCAL

CUÁNDO SIRVE

- Versiones publicadas y soportadas en paralelo
- Software instalable en el cliente
- Ventanas de publicación planificadas

QUÉ CUESTA

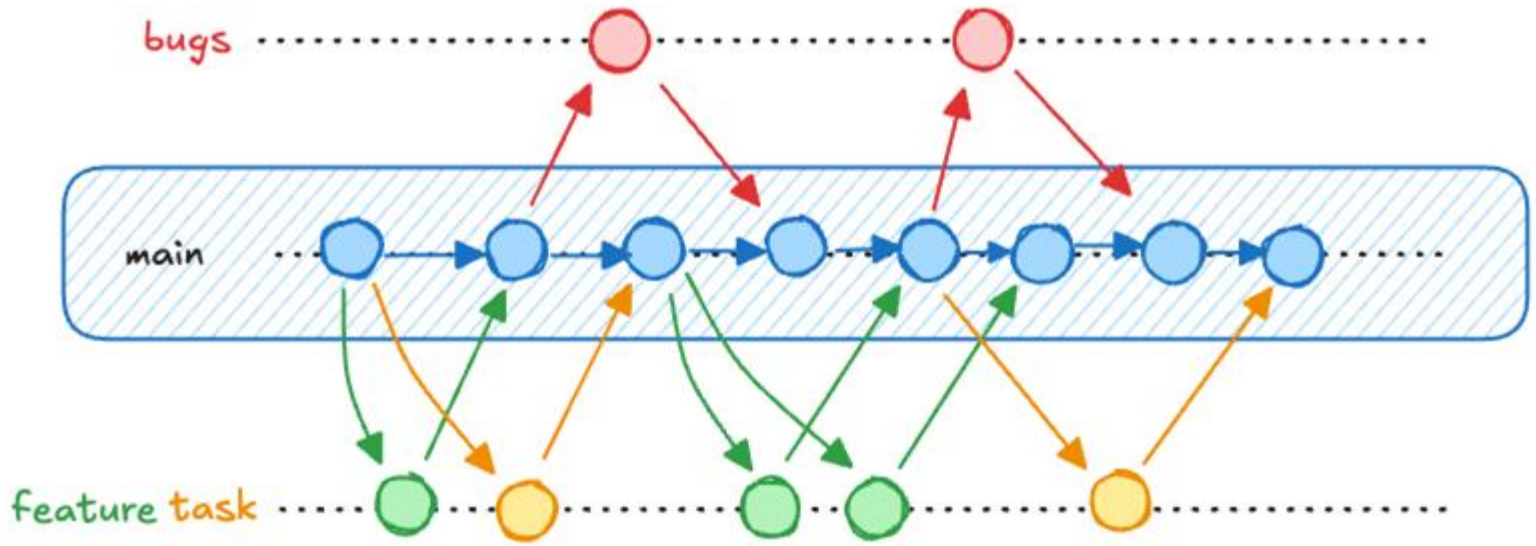
- Ramas de vida larga por diseño
- Conflictos frecuentes y caros

EL MALENTENDIDO MÁS COMÚN

GitFlow se popularizó como el modelo por defecto para cualquier proyecto, incluidos servicios web de despliegue continuo. Su autor aclaró después que fue diseñado para software con versiones publicadas explícitamente, no para aplicaciones que se despliegan varias veces al día.

Usarlo en un servicio web con despliegue continuo es la causa más frecuente de ramas de tres semanas.

Trunk Based



ESTRATEGIA DE RAMIFICACIÓN 2

Trunk Based Development (Basado en troncos)

Todo el equipo integra a una única rama principal. Las ramas de trabajo existen, pero viven horas y no días, y la rama principal se mantiene siempre en estado desplegable.

1

Ramas de vida corta

De horas a un día como máximo. Si una rama va a durar más, el trabajo está mal dividido.

2

Cambios pequeños

Se integra trabajo incompleto pero funcional. Cambia cómo se corta el trabajo.

3

Banderas de funcionalidad (Feature Flag)

El código llega a producción desactivado. La activación es una decisión de producto, no de despliegue.

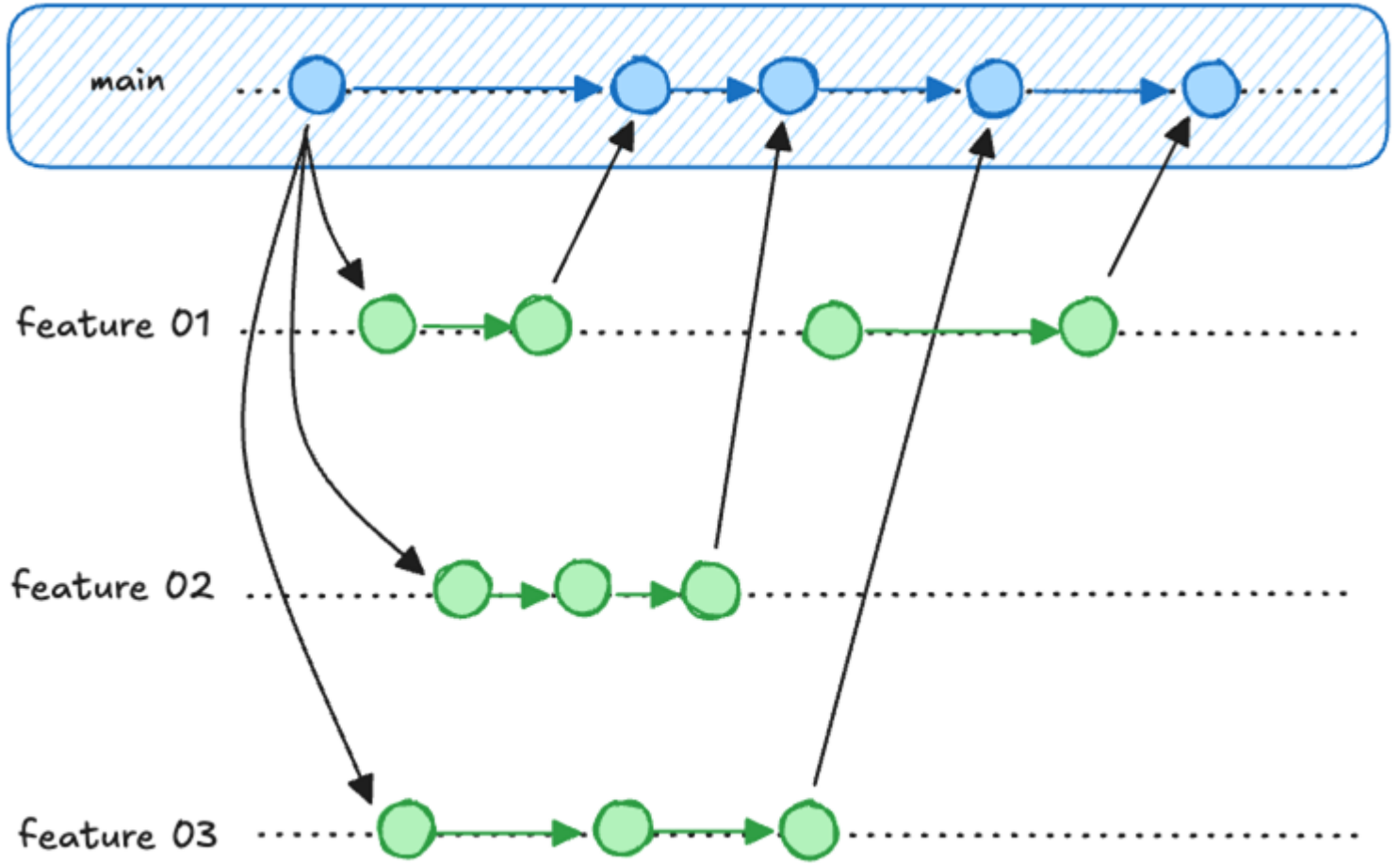
4

Verificación obligatoria

Solo funciona si la verificación automática es rápida y confiable. Sin eso, es integrar caos continuamente.

El punto 4 es la condición que casi nadie cumple antes de intentar el cambio: sin validación automática confiable, trunk based rompe la rama principal todos los días.

Github Flow



CRITERIOS DE ELECCIÓN

Comparación de estrategias

Criterio	GitFlow	GitHub Flow	Trunk Based
Vida de la rama	Semanas	Días	Horas
Frecuencia de integración	Baja	Media	Alta (diaria)
Contexto natural	Versiones soportadas	Producto web	Despliegue continuo
Exige pruebas automáticas	Deseable	Necesario	Indispensable
Exige banderas de funcionalidad	No	A veces	Sí

La mejor estrategia será la que mejor calce con la que tu equipo y proyecto necesite; y sobre todo esté listo para usar.

EL HABILITADOR TÉCNICO

Feature Flags

FORMA MÍNIMA

```
if (flags.isEnabled("nuevo_checkout")) {
    return nuevoCheckout(pedido);
}
return checkoutActual(pedido);
```

1 **Desacopla desplegar de publicar**

El código viaja a producción sin que el usuario lo vea.

2 **Reversión sin despliegue**

Si algo falla, se apaga la bandera. No hay que revertir ni volver a desplegar.

POR QUÉ HABILITA EL TRABAJO EN TRUNK BASED

La objeción "no puedo integrar código incompleto" desaparece: el código incompleto se integra, se prueba junto con todo lo demás, y simplemente no se ejecuta hasta que se decida. Es lo que permite ramas de horas sin exponer trabajo a medias.

COSTOS

- Cada bandera duplica los caminos de ejecución posibles.
- Las banderas que nadie retira se vuelven deuda técnica permanente.
- Una bandera mal manejada puede exponer funcionalidad no probada.

Regla práctica: toda bandera nace con fecha de retiro.

ACTIVIDAD A1 · CONTEXTO

Los tres escenarios

A

Startup, 5 desarrolladores

Aplicación web. Despliegan cuando está listo. Quieren desplegar a diario.

B

Banco peruano, 40 desarrolladores

Aplicación móvil con releases mensuales de fecha fija. Cada versión pasa por dos semanas de certificación de un equipo de QA independiente mientras el desarrollo del siguiente release continúa. Riesgo Operacional exige evidencia auditable de qué versión llegó a producción y quién la aprobó. Tres versiones soportadas en paralelo.

C

Repositorio de IaC

Código fuente de terraform mantenido por 4 personas, solo la rama main se despliega sobre un ambiente. Los cambios pueden demorar algunos días.

1. **Estrategia recomendada**
2. **¿Por qué?**
3. **¿Qué métrica DORA esperan mover?**

Versionado semántico



MAYOR

Cambio incompatible con la versión anterior

MENOR

Funcionalidad nueva compatible hacia atrás

PARCHE

Corrección compatible hacia atrás

POR QUÉ IMPORTA EN EL PIPELINE

- La versión identifica de forma única el artefacto que se despliega.
- Permite responder qué exactamente está corriendo en producción.
- Es la base de la trazabilidad ante una auditoría o un incidente.

DONDE SE ROMPE EN LA PRÁCTICA

Es una convención, no una garantía técnica: depende de que el autor de la librería la respete. Un cambio menor puede romper a sus consumidores si el autor se equivocó al clasificarlo. Aceptar automáticamente cualquier versión menor nueva es confiar en el juicio de terceros sobre qué es compatible.

Convenciones de commit y versionado automático

CONVENCIÓN DE MENSAJE DE COMMIT

```
feat(checkout): agregar validación de monto
fix(auth): corregir expiración de token
feat(api)!: eliminar endpoint v1 de pagos

// feat   -> incrementa MENOR
// fix    -> incrementa PARCHE
// !      -> incrementa MAYOR
```

QUÉ HABILITA

- El pipeline calcula la versión sin intervención humana.
- Las notas de la versión se generan desde el historial.

1 La convención se verifica en el pipeline

Si el mensaje no cumple el formato, la integración falla. La convención que no se verifica no existe.

2 Cada commit trazable a un cambio

Un commit con mensaje "cambios" no permite reconstruir por qué el sistema es como es. Importa en el postmortem.

3 Base de la trazabilidad regulatoria

En un sector regulado hay que demostrar qué cambió, quién lo autorizó y qué validaciones pasó. Empieza acá.

4 Costo real: disciplina de equipo

Barata de implementar y cara de sostener. Solo funciona si está automatizada.

Una convención documentada que no se verifica automáticamente se degrada en dos semanas. Es un patrón general del curso, no solo de commits.

B

Gestión de librerías

El código que no escribimos, los builds reproducibles y el control de lo que entra a la organización

Todos usamos librerías de la comunidad

Muchas de estas librerías llegan de forma indirecta, como dependencia de la librería que sí necesitamos.

CÓMO SE VE EL ÁRBOL

```
mi-aplicación
    └─ framework-web   (declarada)
        └─ analizador-json   (transitiva)
        └─ utilidades-texto   (transitiva)
        └─ cliente-http   (transitiva)
    └─ librería-fechas   (declarada)
        └─ zona-horaria   (transitiva)
```

DEPENDENCIA DECLARADA

La que el equipo eligió, evaluó y escribió en el archivo de configuración. Suelen ser entre 10 y 40 en un proyecto real.

DEPENDENCIA TRANSITIVA

La que llegó porque una dependencia declarada la necesitaba. Nadie del equipo la eligió ni la revisó, y suelen ser cientos o miles. Corre con los mismos privilegios que el código propio.

Una librería transitiva puede leer variables de entorno, abrir conexiones de red y acceder al sistema de archivos igual que el código que ustedes escribieron y revisaron.

CADENA DE SUMINISTRO DE SOFTWARE

Esto es un potencial problema de seguridad

1

Vulnerabilidad heredada

Una falla en una librería transitiva es una falla en su aplicación. El usuario final no distingue de quién era el código.

2

Compromiso del paquete de origen

Si la cuenta de quien publica una librería popular se ve comprometida, el código malicioso llega a todos sus consumidores en la siguiente actualización.

3

Confusión de nombres

Paquetes publicados con nombres muy similares a los legítimos, o que suplantan nombres de paquetes internos de una organización.

4

Dependencias abandonadas

Librerías sin mantenimiento activo que acumulan vulnerabilidades conocidas y que nadie va a corregir.

El NIST SP 800-218 dedica una sección completa a la gestión de componentes de terceros. Es referencia obligatoria del curso y sustento del Capítulo 6.

Mide tu propio proyecto

1. ¿Cuántas dependencias tienes?

```
# declaradas
npm ls --depth=0 | wc -l

# total instaladas
npm ls --all --parseable | wc -l
```

El segundo número menos el primero son las que nadie del equipo eligió.

2. ¿Tienes builds reproducibles?

```
git ls-files | grep -i lock
```

Si no devuelve nada, el archivo de bloqueo no está versionado y el build no es reproducible.

3. ¿Cuánto vive tu rama más vieja?

```
git for-each-ref --sort=committerdate
\
  --format='%(refname:short)
%(committerdate:short)' \
  refs/heads/
```

Según esto qué modelo de branching creen que usan?

Moodle (https://github.com/moodle/moodle)

48 librerías referenciadas

118 librerías en total heredadas

Builds reproducibles y archivos de bloqueo

SIN ARCHIVO DE BLOQUEO

La declaración dice "cualquier versión 4.x". El build de hoy trae la 4.2.1 y el de mañana trae la 4.3.0, publicada anoche por alguien que ustedes no conocen. El mismo commit produce dos artefactos distintos.

CON ARCHIVO DE BLOQUEO

El archivo registra la versión exacta y la huella criptográfica de cada dependencia, declarada y transitiva. El mismo commit produce el mismo artefacto hoy, mañana y dentro de dos años.

1. **El archivo de bloqueo se versiona**
   Va al repositorio junto al código. Un proyecto sin él en control de versiones no tiene builds reproducibles.
2. **En el pipeline se instala desde el archivo de bloqueo**
   El comando de instalación debe respetar el archivo estrictamente, no resolver versiones de nuevo.
3. **Actualizar es una decisión explícita**
   Subir versiones es un cambio que se revisa y se prueba, no algo que ocurre solo cuando alguien vuelve a construir.
4. **Sin reproducibilidad no hay diagnóstico**
   Si no pueden reconstruir el artefacto exacto que falló en producción, el análisis del incidente es una conjetura.

El archivo de declaración expresa intención; el de bloqueo expresa hecho. Los dos van al repositorio y sirven para cosas distintas.

Repositorios de artefactos y proxy interno

Repositorio público

Proxy interno de la organización

Caché y control de versiones

1

Disponibilidad

Si el repositorio público se cae o retira un paquete, la organización sigue construyendo.
Ha detenido despliegues a escala global.

2

Control de entrada

Se puede bloquear una librería con vulnerabilidad conocida o licencia incompatible antes de que entre a cualquier proyecto.

3

Trazabilidad

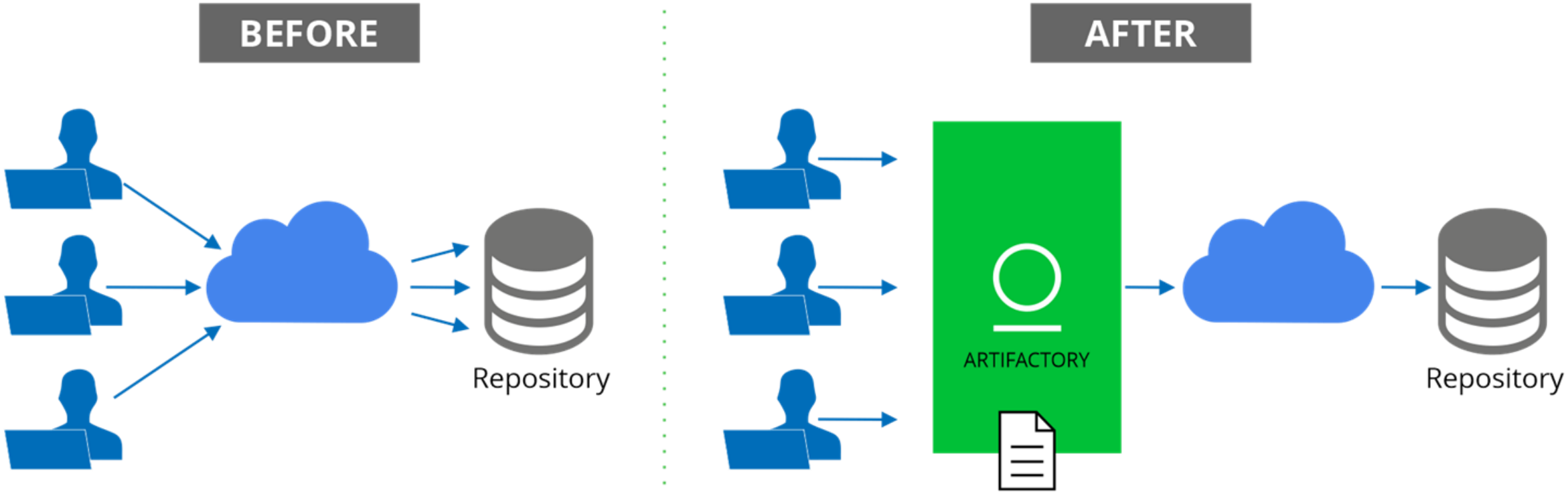
Queda registro de qué componentes externos entraron, cuándo y por qué proyecto. Es insumo directo de auditoría.

4

Artefactos propios

El mismo repositorio almacena los artefactos que la organización construye, versionados e inmutables, listos para desplegar.

Repositorios de artefactos y proxy interno



C

Build

Construir una vez, desplegar muchas: el artefacto inmutable y el tiempo de ejecución como métrica

Construir una vez, desplegar muchas

El artefacto se construye una sola vez, en el pipeline, y ese mismo artefacto binario recorre todos los ambientes hasta producción. La configuración cambia por ambiente; el artefacto no.

ANTI-PATRÓN: RECONSTRUIR POR AMBIENTE

- Se construye una vez para QA y otra vez para producción.
- Entre ambas construcciones cambió una dependencia transitiva.
- Lo que se probó no es lo que se desplegó.
- El defecto aparece solo en producción y nadie puede reproducirlo.
- Se pierde la única garantía que ofrece el pipeline.

1. **Artefacto inmutable**
   Una vez construido no se modifica. Si hace falta un cambio, se construye una versión nueva.
2. **Configuración externa**
   Credenciales y parámetros por ambiente se inyectan en el despliegue, no se compilan dentro.
3. **Identificado por versión**
   Cada artefacto lleva su versión y la referencia del commit que lo originó.
4. **Almacenado, no reconstruido**
   Para desplegar a producción se descarga el artefacto ya validado.

Lo que se probó tiene que ser exactamente el mismo binario que llega a producción.

Anatomía de un trabajo de construcción

GITHUB ACTIONS — TRABAJO DE CONSTRUCCIÓN

```yaml
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 20      # 1
          cache: npm            # 3

      - name: Instalar desde el bloqueo
        run: npm ci             # 2

      - name: Construir
        run: npm run build

      - name: Publicar artefacto
        uses: actions/upload-artifact@v4
        with:
          name: app-${{ github.sha }}    # 4
```

1 **Versión fija del entorno**

La versión del intérprete se declara. "La que tenga la máquina" no es reproducible.

2 **Instalación desde el bloqueo**

El comando respeta el archivo de bloqueo estrictamente. Este detalle es la reproducibilidad.

3 **Caché de dependencias**

Reduce minutos por ejecución. Impacta directamente el lead time

4 **Artefacto identificado**

El nombre incluye la referencia del commit. Trazabilidad de artefacto a código fuente.

IMPACTO SOBRE EL FLUJO

El tiempo de construcción es una métrica de proceso
Si es muy alta, los developers evitarán el pipeline

3 min

se queda esperando y corrige de inmediato

10 min

límite de tolerancia

20 min

se va a hacer otra cosa y vuelve después

REGLA DE DISEÑO

El pipeline de integración debe responder en menos de 10 minutos. Si toma más, los desarrolladores empiezan a evitarlo: agrupan cambios, integran menos seguido y la práctica se degrada sola.

Qué mide el análisis automático de calidad

Conjunto de reglas que se ejecutan sobre el código fuente sin correrlo, más las métricas que produce la ejecución de las pruebas. Todo se resume en números comparables entre una ejecución y la siguiente.

LO QUE SE MIDE

- **Cobertura**: qué porción del código ejecutan las pruebas unitarias.
- **Duplicación**: líneas repetidas dentro del proyecto.
- **Complejidad**: caminos de ejecución de una función.
- **Issues** por severidad, detectados sin correr el código.
- **Deuda técnica**: tiempo estimado de corrección acumulado.

LO QUE NO SE MIDE

- Si la prueba verifica algo que importa o solo recorre líneas.
- Si el diseño de la solución es adecuado al problema.
- Si el requisito se entendió bien antes de programarlo.
- Si el sistema hace lo que el negocio necesita.
- Si la arquitectura sostiene el crecimiento previsto.

Sirve para detener lo evidentemente malo, no para certificar lo bueno.

Quality Gate: del reporte a la condición

Condición declarada que el resultado del análisis debe cumplir para que el pipeline continúe. Si no se cumple, el paso falla y el artefacto no se publica.

ANÁLISIS QUALITY GATE DESACTIVADO

El pipeline analiza, publica un reporte y continúa. El número empeora cada mes y nadie lo detiene, porque no existe ningún momento en que detenerlo sea obligatorio.

ANÁLISIS QUALITY GATE ACTIVADO

El resultado se compara contra una condición escrita antes de conocer el hallazgo. La decisión de qué es inaceptable ya está tomada cuando aparece el caso incómodo.

1

Condición explícita y versionada

Las reglas y los umbrales viven en el repositorio, no en el criterio de quien revisa ese día.

2

Resultado binario

El paso va o no va.

3

Detiene, no advierte

Si no detiene el pipeline, el análisis es un reporte. Depende de la madurez de los equipos.

Sin quality gate, el análisis de calidad es documentación. Con gate, es un control. La diferencia es un equipo listo para trabajar de esa manera.

DISEÑO DEL UMBRAL

El umbral que nadie puede pasar se termina desactivando

UMBRAL SOBRE TODO EL CÓDIGO

Un proyecto con años de historia no alcanza un umbral alto de cobertura global. El equipo queda con dos salidas: dedicar meses a probar código que nadie va a tocar, o desactivar el gate. Elige la segunda, y el gate desaparece del pipeline para siempre.

UMBRAL SOBRE EL CÓDIGO NUEVO

La condición se evalúa solo sobre las líneas que este cambio agregó o modificó. El proyecto mejora de forma incremental, el equipo puede cumplir la condición desde el primer día, y lo heredado deja de ser una excusa para no tener control.

1

Partir del valor actual

El umbral inicial es la medición de hoy. Lo único que se exige al principio es no empeorar.

2

Subirlo por acuerdo

Un umbral que el equipo no aceptó se vuelve la excepción permanente que todos aprenden a saltar.

3

Excepción con justificación

Toda exclusión de una regla se escribe en el repositorio. Sin razón escrita, es un hallazgo silenciado.

Los umbrales se setean inicialmente y evolucionan junto con el equipo.

Anatomía de la validación en el pipeline

GITHUB ACTIONS — VALIDACIÓN Y QUALITY GATE

```
jobs:
  validar:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - name: Instalar desde el bloqueo
        run: pip install -r requirements.lock
      - name: Pruebas con reporte de cobertura
        run: pytest --cov=src --cov-report=xml
      - name: Analisis de calidad y umbral
        uses: <accion-del-servicio-de-analisis>
        env:
          TOKEN: ${{ secrets.TOKEN_ANALISIS }}

  desplegar:
    needs: validar
    runs-on: ubuntu-latest
    steps:
      - name: Publicar la funcion en AWS
        run: aws lambda update-function-code ...
```

1

Historial completo

Sin el historial, el análisis no distingue el código nuevo del preexistente.

2

La cobertura es un insumo

Las pruebas generan el reporte que el analizador consume. Sin reporte, cobertura cero.

3

Aquí está el quality gate

El despliegue declara que necesita la validación. Si falla, no llega a ejecutarse.

4

Credenciales como secreto

Nunca escritas en el archivo.

Kahoot de cierre

Viene con puntos adicionales:
1er puesto: +1 punto en parcial
2do puesto: +0.5 en parcial
3er puesto: +0.5 en parcial

CIERRE SESIÓN 3

Que se llevan

1

Integrar seguido

Integrar seguido no es solo evitar conflictos, es adelantar posibles fallas en el proceso

2

La estrategia branching se elige por capacidades

Recomendar trunk based a un equipo sin pruebas automatizadas es recomendar caos continuo.

3

El código que no escribimos también es nuestro

Las librerías heredadas por nuestras dependencias corren con los mismos privilegios que el código propio.

4

Construir una vez, desplegar muchas

Reconstruir por ambiente destruye la única garantía que ofrece el pipeline.

5

El tiempo de ejecución es condición de adopción

Un pipeline de más de 10 minutos se empieza a evitar, y deja de proteger.

LABORATORIO 2 · PRESENTACIÓN

Pipeline con validación de calidad

QUÉ VAN A CONSTRUIR

Un pipeline que ejecuta pruebas unitarias con reporte de cobertura, aplica el análisis de calidad con un umbral declarado, y despliega una función en AWS únicamente si el umbral se cumple.

EJERCICIO DE INYECCIÓN DE FALLA

Parte de la nota consiste en romper deliberadamente el umbral y demostrar con evidencia que el pipeline se detuvo y que el artefacto no llegó a desplegarse. Un pipeline que solo se ha visto pasar no está probado.

RÚBRICA — 20 PUNTOS

Pipeline ejecuta pruebas con reporte de cobertura	3
Análisis de calidad integrado y umbral configurado	4
Despliegue de la función condicionado al umbral	2
Evidencia de la inyección de falla	3
Preguntas de reflexión sobre el diseño del umbral	8